

AI 투자챗봇 전체 구조 설명서

Spring Boot + Python AI Agent + RAG 통합 구조

1. 현재 script.py 구조

현재는 단순 RAG 챗봇 구조입니다. 사용자 질문이 들어오면 ChromaDB에서 관련 문서를 검색하고 Gemini가 답변을 생성합니다.

단계	설명
Spring Boot	사용자 질문 수신 및 script.py 호출
ChromaDB	벡터DB에서 관련 법령 문서 검색
Gemini LLM	검색된 문서 기반 답변 생성
stdout 출력	Spring Boot로 답변 반환 → MySQL 저장

2. 목표 script.py 구조 (Agent)

앞으로는 Agent가 Tool을 직접 선택하고 실행하는 구조로 전환됩니다.

단계	설명
Spring Boot	사용자 질문 수신
ReAct Agent	질문 분석 → 필요한 Tool 판단
RAG Tool	법령 PDF 벡터DB 검색
네이버 크롤링	실시간 뉴스/주가 수집
한투 API	실시간 주가, 재무제표, 채권/선물
Gemini LLM	수집된 정보 종합 분석

3. 핵심 코드 변경 포인트

기존 RAG 호출 코드:

```
response = chain.invoke(question)
```

Agent 호출로 변경:

```
from AI_Agent.agent import run_agent
response = run_agent(question)
```

4. Agent 동작 예시

예시 1: '삼성전자 왜 떨어졌어?'

단계	동작
Thought	삼성전자 현재가 확인 필요
Action	naver_finance_crawler 실행
Thought	관련 뉴스 확인 필요
Action	naver_news_crawler 실행
Thought	뉴스 기반 원인 분석
Final Answer	반도체 수출 감소, 외국인 순매도, TSMC 경쟁 심화

예시 2: '미래에셋 펀드 설명해줘'

단계	동작
Thought	PDF 설명서 필요
Action	RAG_Search 실행
Final Answer	펀드 내용 기반 답변

예시 3: 'SK 하이닉스 재무제표 보여줘'

단계	동작
Thought	재무 데이터 필요
Action	한투 API 실행
Final Answer	재무제표 표 형태로 출력

5. 프로젝트 최종 폴더 구조

```
OHSUNGJUN_CHAT/  
├─ AI_Agent/  
│   ├── agent.py           ← ReAct Agent 뇌  
│   ├── tool_registry.py   ← 도구 메뉴판  
│   ├── cache_manager.py   ← API 비용 절약  
│   └─ tools/  
│       ├── Financial_product.py ← 금융상품 코드  
│       ├── naver_finance_crawler.py  
│       ├── naver_news_crawler.py  
│       ├── korea_investment_api.py  
│       └─ table_maker.py  
├─ RAG/  
│   ├── chroma_db/  
│   └─ build_chroma1.py  
├─ script.py               ← Spring Boot 진입점  
└─ communication.py       ← 터미널 테스트
```

6. 이 구조의 장점

강점	설명
환각 감소	실제 데이터(API/크롤링) 기반 답변
최신 데이터	실시간 주가, 뉴스 반영
법령 기반	RAG로 정확한 규정 답변
확장성	Tool만 추가하면 기능 확장 가능
실무형 구조	핀테크 AI 서비스 구조와 동일